KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

CLINIC PRESCRIPTION RECORD AND PATIENT VISIT TRACKING SYSTEM

Submitted by
NANDHINI R
25BIT033

Under the Guidance of
DHILEEBAN A
Technical Trainer
Technical Department, KG College of Arts and Science.

Academic Year: 2025 – 2026

APRIL 2026

Semester: 2

DECLARATION

I hereby declare that the project entitled **"CLINIC PRESCRIPTION RECORD AND PATIENT VISIT TRACKING SYSTEM"**, submitted in partial fulfilment of the requirements for Java Programming, is a record of my project work carried out under the guidance of Dhileeban A, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this report has not been submitted previously, either in part or in full, for the award of any degree, diploma, or similar title at this or any other institution.

I have acknowledged the sources of information used in this project wherever applicable.

April 2026
Coimbatore

NANDHINI R
(25BIT033)

CERTIFICATE

This is to certify that the project entitled **"CLINIC PRESCRIPTION RECORD AND PATIENT VISIT TRACKING SYSTEM"** is a bona fide work carried out by NANDHINI R, B.Sc. Information Technology, KG College of Arts & Science, in partial fulfillment of the requirements for Java Programming.

The project has been completed under my supervision and guidance during the academic period April 2026.

Place: Coimbatore
Date: _____

_____	_____
Guide	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page
1	Introduction	6
1.1	Objective	6
1.2	Overview	6
1.3	Features	6
2	Program Analysis	7
2.1	Problem Definition	7
2.2	Project Overview	7
2.3	Scope and Limitations	7
3	System Design	8
3.1	Modules	8
4	Methodology	9
4.1	Problem Analysis	9
4.2	System Design	9
4.3	Program Development	9
4.4	Testing and Validation	10
4.5	Result and Documentation	10
4.6	Algorithm	10
4.7	Flowchart	11
5	Source Code	12-14
6	Input and Output Screens	15
7	Discussion and Conclusion	16
7.1	Discussion	16
7.2	Conclusion	16
8	References	17

ABSTRACT

The Clinic Prescription Record and Patient Visit Tracking System is a Java-based application designed to organize basic patient details, record clinic visits, and maintain prescription notes. Manual record keeping can lead to misplaced files, repeated data entry, and difficulty retrieving previous visits. This project demonstrates a simple computerized approach to storing and viewing these records.

The application allows clinic staff to register a patient with a patient ID, name, age, and gender. It can record each visit with the visit date, symptoms, doctor name, and prescription notes. Users can view registered patients and retrieve visit history using a patient ID.

The project demonstrates Core Java concepts including classes, objects, ArrayList collections, methods, loops, conditional statements, and console input. It is an educational prototype and does not replace professional clinical judgment or a secure electronic health record.

The main objective is to improve record organization and demonstrate how Java can be applied to a practical clinic record-management task.

1. INTRODUCTION

Clinics need reliable ways to maintain patient information and document visits. Patient details, symptoms, consultation dates, and prescription notes may be written on paper or stored in separate files. Finding older records can take time, especially when a patient visits repeatedly.

A computerized patient record system helps staff organize basic information and retrieve records when required. A patient visit tracking feature creates a history of consultations associated with each patient. Prescription notes can be recorded alongside visit details so the clinic can review what was documented previously.

This Java application supports patient registration, visit recording, viewing all registered patients, and displaying a patient's visit history. It uses in-memory collections, so records are available only while the program is running.

The project manages records only; it does not diagnose illness or recommend treatment. Prescription details should be entered and reviewed by authorized clinical personnel.

1.1 OBJECTIVE

The main objective is to develop a simple Java application that organizes patient registration, prescription notes, and visit history in a clinic environment.

Specific Objectives

- Register patients using a patient ID and basic demographic details.
- Record visit date, reported symptoms, doctor name, and prescription notes.
- Display registered patient records in an organized format.
- Retrieve visit history using the patient's ID.
- Reduce the effort required to search handwritten records.
- Demonstrate Core Java concepts through a practical project.
- Provide a foundation for future database storage and secure access control.

1.2 OVERVIEW

The system starts with a menu that lets the user choose an operation. Patient registration collects ID, name, age, and gender. Visit recording captures the patient ID, date, symptoms, doctor name, and prescription notes. Patient and visit objects are stored in ArrayList collections during execution.

When the user selects the patient list option, the application displays registered patients. When visit history is requested, the user enters a patient ID and the system displays visit records matching that ID.

1.3 FEATURES

- **Patient Registration:** Stores ID, name, age, and gender.
- **Visit Recording:** Captures visit date, symptoms, doctor, and prescription notes.
- **Patient List:** Displays patients registered in the current session.
- **Visit History:** Shows visits associated with a patient ID.
- **Menu-Driven Operation:** Provides a simple text interface.
- **Modular Structure:** Separates patient and visit information into classes.

2. PROGRAM ANALYSIS

Program analysis identifies the problem, expected behavior, and functions needed to meet the project objectives. The main focus is replacing scattered manual notes with a structured way to enter and retrieve patient visit information.

2.1 PROBLEM DEFINITION

Manual clinic records may be difficult to search, can be misplaced, and may not provide a quick overview of previous consultations. Rewriting patient details for every visit creates extra work. If records are not organized consistently, staff may have difficulty locating a prescription note or confirming a previous visit date.

Problems Identified

- Time-consuming retrieval of older patient records.
- Risk of incomplete or inconsistent handwritten details.
- Difficulty tracking repeat visits for the same patient.
- Difficulty viewing prescription notes alongside visit details.
- Physical files require storage space and careful handling.

2.2 PROJECT OVERVIEW

The application is developed in Java and runs in a console. It presents a numbered menu, reads the user's choice, and calls the relevant method. Patient objects are stored in an ArrayList of Patient objects, while visit objects are stored in an ArrayList of Visit objects.

Each visit record includes a patient ID. This ID links a visit to a patient without copying all patient information into every visit. For visit history, the program compares the requested ID with the patient ID saved in each visit.

2.3 SCOPE AND LIMITATIONS

Scope

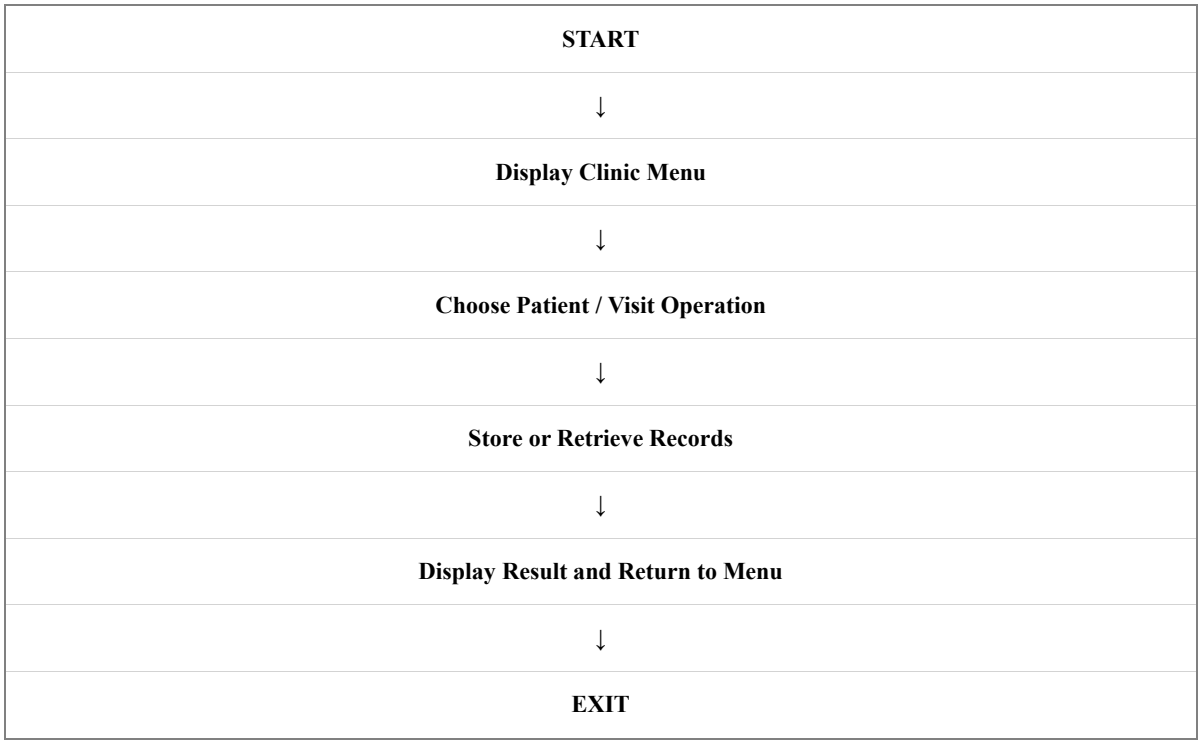
- Register and display basic patient details.
- Record visit date, symptoms, doctor name, and prescription notes.
- View visit history for a selected patient.
- Demonstrate a basic clinic record workflow using Java.

Limitations

- Records are lost when the program closes.
- No login, role permissions, encryption, or audit logging is implemented.
- The prototype does not validate prescription safety.
- No appointment scheduling, billing, laboratory integration, or backup.
- Do not enter real patient data into this educational prototype.

3. SYSTEM DESIGN

System design defines the components of the application and how data moves through them. The program is divided into a main menu, patient registration, visit recording, patient display, visit-history search, and data classes.



3.1 MODULES

1. Patient Registration Module

This module asks for patient ID, name, age, and gender. It creates a Patient object and adds it to the patient collection. A production system should validate inputs and enforce unique IDs.

2. Visit Recording Module

This module collects patient ID, visit date, symptoms, doctor name, and prescription notes. It creates a Visit object and stores it in the visit collection. A future version should verify that the patient exists before accepting a visit.

3. Patient Display Module

This module loops through registered patients and displays their details. If no patient has been registered, it displays an empty-list message.

4. Visit History Module

This module accepts a patient ID, searches the visit collection, and displays each matching visit. If no matching visit is found, it informs the user.

5. Exit Module

This module ends the application when the user selects the exit option.

4. METHODOLOGY

The project follows a basic software-development workflow. Requirements are identified, the program structure and data classes are designed, Java source code is implemented, and the system is tested with sample inputs.

4.1 PROBLEM ANALYSIS

The requirements are based on common clinic record-management needs: registering a patient, recording a consultation, retaining prescription notes for the visit, and retrieving patient history. The application is limited to these basic functions for a beginner-level Java project.

Functional Requirements

- The user can add a patient.
- The user can record a visit with prescription details.
- The user can view all patients.
- The user can search visit history using a patient ID.
- The system displays a message when a list or search result is empty.
- The user can exit from the menu.

4.2 SYSTEM DESIGN

The application uses two main data classes: Patient and Visit. Patient stores patient ID, name, age, and gender. Visit stores patient ID, visit date, symptoms, doctor name, and prescription notes. The main class manages the menu and collections.

4.3 PROGRAM DEVELOPMENT

The program uses Core Java. Scanner reads console input, ArrayList stores objects during execution, and a switch statement routes menu choices to relevant methods. A loop keeps the menu running until the user chooses to exit.

- Create Patient and Visit classes with constructors.
- Create ArrayList collections to hold objects.
- Build menu options for registration, visit recording, patient listing, and history.
- Read user input and create the appropriate object.
- Search visit records by patient ID and display matches.
- Handle invalid menu choices with a message.

4.4 TESTING AND VALIDATION

Testing checks whether each menu operation behaves as expected for typical and empty-result scenarios.

Test Case	Input / Action	Expected Result
Register patient	Enter patient details	Patient is stored and success message appears
Record visit	Enter visit details	Visit recorded message
View patients	Choose patient list	Registered patients displayed
Visit history	Enter matching ID	Matching visits displayed
Unknown ID	Search ID with no visits	No visit records message
Invalid choice	Enter unsupported number	Invalid choice message

Further validation should include duplicate IDs, invalid ages, missing fields, and visits linked to nonexistent patients.

4.5 RESULT AND DOCUMENTATION

The program provides a menu-based way to enter patient details, record consultation information, list registered patients, and view visit history. Output is designed to make entered information readable within the current program session.

The documentation explains the problem, objectives, system modules, development process, algorithm, source code, and sample console interaction. Screenshots for a final submission should be captured from the actual program execution.

4.6 ALGORITHM

1. Start the program.
2. Create empty patient and visit collections.
3. Display the clinic management menu.
4. Read the user's menu choice.
5. If Add Patient is selected, read details and store a Patient object.
6. If Record Visit is selected, store visit and prescription details.
7. If View Patients is selected, display registered patients.
8. If Visit History is selected, search by patient ID.
9. Display a message for empty results or invalid choices.
10. Repeat until Exit is selected.
11. End the program.

4.7 FLOWCHART:

START
↓
Initialize patient and visit lists
↓
Display menu and read choice
↓
Select operation: Add / Record / View / History
↓
Process input and display result
↓
Return to menu unless Exit selected
↓
END

5. SOURCE CODE

Save the following code as **ClinicManagement.java**. It is a console-based educational prototype. Data is stored in memory only.

```
import
java.util.ArrayList;
import java.util.Scanner;

class Patient {     int id, age;     String name, gender;
Patient(int id, String name, int age, String gender) {
this.id=id; this.name=name; this.age=age;
this.gender=gender;     }
   void displayPatient() {
System.out.println("Patient ID: " + id);
       System.out.println("Name: " + name);
       System.out.println("Age: " + age);
       System.out.println("Gender: " + gender);
       System.out.println("------------------------");
   }
}

class Visit {     int patientId;     String date, symptoms, doctor,
prescription;     Visit(int id, String date, String symptoms, String
doctor, String prescription) {         this.patientId=id;
this.date=date; this.symptoms=symptoms;
this.doctor=doctor; this.prescription=prescription;     }
   void displayVisit() {
System.out.println("Patient ID: " +
patientId);
       System.out.println("Visit Date: " + date);
       System.out.println("Symptoms: " + symptoms);
       System.out.println("Doctor: " + doctor);
       System.out.println("Prescription Notes: " +
prescription);
       System.out.println("------------------------");
   }
}
```

5. SOURCE CODE (CONTINUED)

```
public class ClinicManagement {     static
ArrayList<Patient> patients = new
ArrayList<>();     static ArrayList<Visit>
visits = new ArrayList<>();

   public static void main(String[] args) {         Scanner sc =
new Scanner(System.in);         while (true) {
System.out.println("\n--- Clinic Prescription & Visit System
---");
          System.out.println("1. Add Patient");
          System.out.println("2. Record Visit / Prescription");
          System.out.println("3. View All Patients");
          System.out.println("4. View Patient Visit History");
          System.out.println("5. Exit");
System.out.print("Enter choice: ");             int choice;
try { choice = Integer.parseInt(sc.nextLine().trim()); }
catch (NumberFormatException e) {
System.out.println("Please enter a valid number.");
continue;            }            switch (choice) {                case
1: addPatient(sc); break;                case 2: recordVisit(sc);
break;                case 3: viewPatients(); break;
case 4: viewVisitHistory(sc); break;
              case 5: System.out.println("Exiting...");
sc.close(); return;                default:
System.out.println("Invalid choice!");            }
      }
}

   static int readInt(Scanner sc) {         try {
return Integer.parseInt(sc.nextLine().trim()); }
catch (NumberFormatException e) {
System.out.println("Invalid number."); return -
1;
      }
   }
```

5. SOURCE CODE (CONTINUED)

```
   static void addPatient(Scanner sc) {
System.out.print("Enter Patient ID: "); int
id=readInt(sc);         if (id < 0) return;
       for (Patient p : patients) if (p.id == id) {
System.out.println("Patient ID already exists."); return;
       }
       System.out.print("Enter Name: "); String name=sc.nextLine();
System.out.print("Enter Age: "); int age=readInt(sc);         if (age < 0) return;
System.out.print("Enter Gender: "); String gender=sc.nextLine();
patients.add(new Patient(id,name,age,gender));         System.out.println("Patient
added successfully!");     }

   static void recordVisit(Scanner sc) {
System.out.print("Enter Patient ID: "); int id=readInt(sc);
if (id < 0) return;         boolean exists=false;         for
(Patient p : patients) if (p.id == id) exists=true;         if
(!exists) { System.out.println("Patient ID not found.");
return; }         System.out.print("Enter Visit Date: "); String
date=sc.nextLine();
       System.out.print("Enter Symptoms: "); String
symptoms=sc.nextLine();
       System.out.print("Enter Doctor Name: "); String
doctor=sc.nextLine();         System.out.print("Enter
Prescription Notes: "); String rx=sc.nextLine();
visits.add(new Visit(id,date,symptoms,doctor,rx));
System.out.println("Visit recorded successfully!");     }

   static void viewPatients() {         if (patients.isEmpty()) {
System.out.println("No patients found."); return; }         for
(Patient p : patients) p.displayPatient();     }

   static void viewVisitHistory(Scanner sc) {
System.out.print("Enter Patient ID: "); int id=readInt(sc);         if
(id < 0) return; boolean found=false;         for (Visit v : visits) if
(v.patientId == id) { v.displayVisit(); found=true; }         if (!found)
System.out.println("No visit records found for this patient.");     }
}

// Educational prototype only. Do not store real patient
data.
// Production use requires secure storage, access control, backups, and audit
logs.
```

6. INPUT AND OUTPUT SCREENS

The following sample console session illustrates expected interaction. If actual screenshots are required, capture them from the program running in your Java environment.

Sample Input: Register a Patient

```
--- Clinic Prescription & Visit System ---
1. Add Patient
2. Record Visit / Prescription
3. View All Patients
4. View Patient Visit History
5. Exit
Enter choice: 1
Enter Patient ID: 101
Enter Name: Example Patient
Enter Age: 28
Enter Gender: Female
Patient added
successfully!
```

Sample Input: Record a Visit

```
Enter choice: 2
Enter Patient ID: 101
Enter Visit Date: 26-10-2026
Enter Symptoms: Cough and sore throat
Enter Doctor Name: Dr. Example
Enter Prescription Notes: As recorded by
clinician Visit recorded successfully!
```

The sample details are fictional and are included only to demonstrate the program flow.

7. DISCUSSION & CONCLUSION

7.1 DISCUSSION

The Clinic Prescription Record and Patient Visit Tracking System demonstrates how a simple Java application can organize patient registration and visit documentation. The menu-driven interface provides a clear set of operations, while Patient and Visit classes keep the two types of records logically separated.

The visit-history feature shows how a patient ID can connect multiple consultations to one patient. Collections allow the program to handle multiple objects during one run. Testing should include empty collections, matching and non-matching IDs, invalid menu choices, and incorrect numeric input.

The prototype has limitations: data is not saved permanently and authentication, permissions, encryption, and audit logs are not implemented. It must be treated as an educational demonstration, not a production clinical record system.

7.2 CONCLUSION

The project demonstrates patient registration, visit recording, prescription-note storage, patient listing, and visit-history retrieval using Core Java. It reduces the need to manually search records within the current program session and illustrates practical object-oriented programming.

Future enhancements may include database connectivity, secure login, role-based permissions, permanent storage, backup and recovery, date-based search, report generation, and a graphical user interface. Real deployment requires appropriate privacy, security, and clinical governance measures.

8. REFERENCES

The following resources provide background information on Java programming and the tools used to develop a basic console application.

Books

- Herbert Schildt, *Java: The Complete Reference*.

Websites

- Oracle Java documentation: https://docs.oracle.com/en/java/
- GeeksforGeeks Java tutorials: https://www.geeksforgeeks.org/java/
- W3Schools Java tutorial: https://www.w3schools.com/java/
- Javatpoint Java tutorials: https://www.javatpoint.com/java-tutorial

Note: This project is an educational prototype. It is not a substitute for a certified electronic health record system and should not be used to store identifiable real patient information without appropriate security and authorization.